Sterl: Banking App Plan for JPMorgan

Nov 2, 2026 · updated Nov 5, 2026 · @Mehak Javed

What JPMorgan wants from a React developer

JPMorgan's London React roles are mostly in the Commercial & Investment Bank, building trading and markets screens. They ask for React + TypeScript, testing, data grids and fast real-time updates.

Skill	How JPMorgan's London postings phrase it	Your sprint week
React + TypeScript	Required in every posting reviewed; "production web applications with React and TypeScript"	Already have; deepen in weeks 1–2
Testing	Jest and React Testing Library listed as required	Week 3
State management + async data	Redux and hooks; "state management and asynchronous data flows"	Week 1
Real-time, high-frequency data	Live trading screens with "high-frequency updates"	Week 3 (performance)
Data grids	AG Grid or similar advanced grids, preferred	Week 3
Performance	Profiling, tuning, browser performance	Week 3
Security	Secure code and awareness of web security risks	Weeks 2–3, then backend phase
CI/CD + observability	CI/CD pipelines, logging, metrics, alerting, incident triage	CI from week 1; observability in week 4
AI coding tools	Using approved AI assistants and checking their output for correctness and security	Every week
Finance knowledge	Trading, orders, FX or futures & options exposure, preferred	Learn while building Sterl
Java + SQL	"One or more extra programming languages", preferred in React roles; **required** in Java / React / AI roles, with "database querying languages"	Week 2 onwards

Experience reality check: many of these roles (Software Engineer III and above) expect several years of professional experience, often in a large corporate environment. A project can't replace that, but it makes you stand out. Apply to these roles *and* to JPMorgan's early-career routes (graduate Software Engineer Program, Code for Good hackathon), plus other UK banks and fintechs.

Target role: Software Engineer III – Java / React / AI NEW

JPMorganChase, **Wholesale Credit Risk**, Glasgow. This posting puts **Java first**, then React, then AI-assisted engineering, all inside a credit risk business. SterI now covers every line of it.

What the posting asks for	How SterI shows it	When
Java + a database querying language	Spring Boot API with PostgreSQL, hand-written SQL for reports, JUnit tests	Week 2 onwards
Wholesale Credit Risk domain	The Credit Risk desk: fake corporate borrowers, ratings, limits, exposure and breach alerts	Week 3
Secure, high-quality production code	Integer-pence money, input validation (Zod, Bean Validation), secrets kept out of git, security pass	Every week
Algorithms that run in step with other systems	Exposure and limit-utilisation calculations in Java, shared by the API and reports; idempotent payments	Weeks 2–3
AI coding tools, with outputs validated	`AI_USAGE.md` log, pull request checklist, every AI-written change covered by tests	Every week
Responsible AI: data sensitivity, secure inputs/outputs	Ask SterI guardrails: fake data only, field allow-list, prompt-injection checks, answers checked against SQL	Week 4
SDLC toolchain + automation	GitHub Actions CI from week 1: lint, typecheck, tests, build for both front end and Java	Week 1
Architecture and design artifacts	ADRs (architecture decision records) and a system diagram in `docs/`	Weekly
Visualisations and reporting from large datasets	Credit risk dashboard over thousands of fake facilities: AG Grid, charts, CSV export	Week 3
Find hidden problems and patterns in data	Limit-breach and concentration alerts, subscription detector, fraud alerts	Weeks 3–4
Testing and operational stability	Vitest, React Testing Library, Playwright, JUnit; health checks, logs, error tracking	Weeks 1–4
Agile: CI/CD, application resiliency, security	GitHub Projects board with user stories, branches, pull requests, releases; retries, fallbacks, idempotency	Every week
Modern front-end (preferred)	Next.js, React, TypeScript, TanStack Query, Zustand, Redux Toolkit	Weeks 1–3
Cloud (preferred)	Java API + PostgreSQL deployed to AWS (free tier, with a budget alert); front end on Vercel	Week 4

Your qualifications: no extra certification needed

"Formal training or certification on software engineering concepts" is covered by your **MSc in Computer Science** and your work as a software engineer. Your **Claude Code certificate** (software engineering and AI) supports the AI-tooling requirements. Cloud is only "preferred", and deploying Sterl to AWS shows it better than another certificate would. Put both qualifications near the top of your CV, next to the Sterl link.

Why Sterl is a website, not a phone app

Sterl is a **responsive web app**: a website that looks and works like a banking app on a phone, and like a full trading desk on a laptop.

Reason	What it means for you
It matches the job	JPMorgan's London React roles build browser-based screens. A web app proves the exact skills they test; a native phone app (Swift, Kotlin, React Native) proves different ones.
Recruiters can open it in seconds	A live link on your CV opens instantly. Nobody has to install anything.
It fits a one-month launch	Deploying to Vercel takes minutes. App store reviews, signing and device testing would use up your month.
Mobile is still covered	A mobile-first layout makes Sterl feel like an app on a phone. Later you can make it installable to the home screen as a PWA.

How a bank sees software

A bank cares less about flashy features and more about trust: money must never be wrong, lost or stolen. Showing you think this way sets you apart from other candidates.

Bank concern	What it means for your code	How Sterl shows it
Money accuracy	Never store money as decimals; use integer pence. Balances must always add up	All amounts stored in pence; tests prove round-ups and totals are exact
Security	Protect logins, sessions and data; stop XSS, CSRF and injection	Re-authentication before sensitive actions; card details hidden until confirmed
Auditability	Regulators need a record of who did what, and when	An audit log of every transfer, card freeze and setting change
Resiliency	The app must cope when a service is slow or down	Clear loading, error and retry states; a payment can't be sent twice

Bank concern	What it means for your code	How Sterl shows it
Real-time performance	Trading screens update many times a second without freezing	Live FX panel streaming thousands of simulated price updates smoothly
Observability	Find and fix problems in production fast	Error tracking and logging hooked up from day one
Accessibility	UK banks must serve every customer, including disabled users	Keyboard-only and screen-reader tested; high-contrast mode
Credit risk NEW	Know how much each borrower owes and never let it pass their limit unnoticed	Exposure calculated in Java, limit breaches flagged and logged
Responsible AI	Use AI tools, but verify their output; protect sensitive data	`AI_USAGE.md` log, tests on AI-written code, Ask Sterl guardrails
Data protection	Never handle real customer data without a legal reason	Every person, account and payment in Sterl is generated fake data

In interviews, mention these trade-offs when you walk through your project. For example: "I used idempotency keys so a double-click can't send a payment twice."

Sterl: the web app

Sterl is a banking web app with three parts: a **personal banking dashboard** inspired by Chase UK, a **live FX markets desk** inspired by JPMorgan's trading screens, and a **Credit Risk desk** for lending to companies. The front end is Next.js; the core banking logic runs in a **Java + Spring Boot API** over PostgreSQL. Together they show the customer-facing, real-time, data-heavy and back-end skills JPMorgan hires for.

Use your own brand, colours and logo, never Chase's or JPMorgan's. Use fake data only (see "Data: 100% fake, no approval needed" below).

What Chase UK does (the reference)

The Chase UK app offers round-ups into a savings account, spending insights with budgets, multiple accounts for budgeting, a numberless card with details in the app, and fee-free spending abroad. Sterl covers these basics, then adds the features below.

Core features

Feature	What it does	Skills it proves
Accounts dashboard	Balances across current, saver and round-up accounts	Server Components, TanStack Query

Feature	What it does	Skills it proves
Transactions	Searchable, filterable list with categories	Data grid, filtering, pagination
Send money	Pay a saved or new (fake) payee with a review step; only moves fake money between fake accounts	React Hook Form + Zod, server actions
Cards	Freeze/unfreeze; reveal card details after re-auth. Card numbers are published test numbers, never real ones	Zustand UI state, security patterns
Round-ups	Round each payment up to the nearest £1 into savings	Exact money maths, unit tests

Innovative features (what makes it stand out)

Feature	The idea	Why JPMorgan would care
Live FX desk	A grid of 30+ currency pairs ticking several times a second, with a price chart and a mock "order" button. Prices start from free daily reference rates, then a simulator makes them tick	Matches their trading UI roles: real-time data, AG Grid, performance
Credit Risk desk NEW	Fake corporate borrowers with credit ratings, credit limits and loans. Shows exposure, limit utilisation, breaches and concentration by sector and rating	Matches the Wholesale Credit Risk team directly: domain knowledge, Java calculations, SQL reporting
Payee name check	Before paying, warns if the name doesn't match the account. Simulated against Sterl's own fake payee list, the same idea as UK Confirmation of Payee (the real service is only for regulated payment firms)	Fraud prevention, a real UK banking rule
Fraud alert simulator	Flags unusual payments (new country, large amount) and asks "Was this you?"	Risk thinking, event-driven UI
Smart round-up rules	Multiply round-ups, or pause them when balance is low	State logic, testable business rules
Subscription detector	Finds repeat payments and shows the yearly cost	Data analysis on transactions
Audit trail page	A timeline of every sensitive action	Auditability, which regulators require
Ask Sterl (AI)	"How much did I spend on food in October?" answered from your fake transactions	Responsible AI integration

Feature	The idea	Why JPMorgan would care
Accessibility mode	High contrast, larger text, full keyboard use	Inclusive design for every customer

The Credit Risk desk NEW

Wholesale credit risk means managing the risk that a **company** the bank has lent to doesn't pay it back. Learn these terms. They're the language of the team.

Term	Plain meaning	In Sterl
Counterparty	The company the bank lends to or trades with	500 fictional companies made with Faker (never real company names)
Credit rating	How likely the company is to repay, from AAA (safest) to CCC (risky)	A rating per counterparty, with a "downgrade" event you can trigger
Facility	A loan or credit line given to a counterparty	2,000+ fake facilities (term loans, revolving credit)
Credit limit	The most the bank will lend to one counterparty	Set per counterparty, stored in pence
Exposure	How much the counterparty owes right now	Sum of drawn amounts across its facilities, calculated in Java
Limit utilisation	Exposure ÷ limit, as a percentage	Colour bands: under 75% fine, 75–100% warning, over 100% breach
Limit breach	Exposure goes over the limit	An alert, an audit-log entry and a "needs review" queue
Concentration risk	Too much lending to one sector, rating or country	Charts of exposure by sector and rating, with thresholds

What you build

- **Data:** extend the seeded generator to create counterparties, facilities and daily drawdowns, then load them into PostgreSQL.
- **Java:** an `ExposureService` that calculates exposure and utilisation with integer maths, plus JUnit tests for the edge cases (zero limit, exactly 100%, a downgrade).
- **SQL reports:** hand-written queries for "top 10 exposures", "breaches this week" and "exposure by sector" (using `GROUP BY`, `JOIN` and window functions).
- **Screens:** an AG Grid of counterparties with utilisation bars, a drill-down page per counterparty, concentration charts and CSV export.

Data: 100% fake, no approval needed

Yes: you can build all of Sterl without FCA authorisation or any other third-party approval. FCA rules apply when you hold real customers' money, read their real bank accounts, or start real payments. Sterl does none of those. It is a **demo bank** that runs on data you generate yourself, plus public exchange rates.

> **What changed in the Nov 4 update:** the payee name check is now simulated against fake payees, card numbers come from published test numbers, FX prices are simulated on top of free public reference rates, the AI feature only ever reads fake data, and the "make it a real bank" phase is now "make it work like a real bank" (still with fake money).

Where each kind of data comes from

Data Sterl needs	Source	Approval or sign-up needed?
Customers, accounts, balances, payees, 500+ transactions	Your own generator using **Faker.js** (`@faker-js/faker`), with a fixed seed so the data is the same every run	None. It's a free npm package
Fake corporate borrowers, facilities and exposures NEW	The same seeded Faker generator, extended for the Credit Risk desk	None
The "bank API" your screens call	Week 1: your own **Next.js route handlers** (`app/api/.../route.ts`). From week 2: your **Java + Spring Boot API** over PostgreSQL, seeded from the same data	None. You write it
Fake API responses in tests	**Mock Service Worker (MSW)**, which intercepts `fetch` calls in tests and returns your fake data	None. It's a free npm package
Real-world FX reference rates (once a day)	**Frankfurter API**, free central-bank reference rates: `https://api.frankfurter.dev/v2/rates?base=gbp`	None. No API key, no account
Live ticking FX prices	Your own **price simulator**, a small random walk around the Frankfurter rate, sent over WebSocket or Server-Sent Events	None. You write it
Card numbers	Published test card numbers (e.g. 4242 4242 4242 4242) that payment companies release for testing	None

Data Sterl needs	Source	Approval or sign-up needed?
Optional extra: "connect a bank" demo	**TrueLayer** or **Plaid** sandbox: mock UK banks with fake logins and fake transactions	A free developer sign-up only, no FCA approval for the sandbox. **Never switch to live mode.**
Ask Sterl (AI)	An LLM API (e.g. Anthropic or OpenAI) reading only your fake transactions	A developer account and API key (pay-as-you-go). No regulator involved. Optional: a rule-based version needs nothing

Step-by-step: getting the fake data into Sterl

Step 1: Install the tools

```
npm install @faker-js/faker
npm install -D msw vitest
```

Step 2: Write a seeded generator in `src/mocks/generate.ts`. A seed means "the same random numbers every time", so your tests and screenshots never change by surprise. Every amount is whole pence.

```
import { faker } from "@faker-js/faker";
faker.seed(42); // same fake data every run

export function makeTransaction(accountId: string) {
  return {
    id: faker.string.uuid(),
    accountId,
    merchant: faker.company.name(),
    category: faker.helpers.arrayElement(["Food", "Transport", "Bills", "Shopping"]),
    amountPence: faker.number.int({ min: 50, max: 15000 }), // £0.50 to £150.00
    date: faker.date.recent({ days: 90 }).toISOString(),
  };
}
```

Step 3: Save it as fixture files. Add a script (`npm run seed`) that writes `accounts.json`, `transactions.json` and `payees.json` into `src/mocks/data/`. Commit these so anyone who clones the repo sees the same bank.

Step 4: Serve it like a real bank API. Create route handlers such as `app/api/accounts/route.ts` and `app/api/transactions/route.ts` that return the JSON. Add a small random delay and an occasional error (e.g. 1 in 20 requests) so you practise loading, error and retry states. Your screens call these with TanStack Query exactly as they would call a real backend.

Step 5: Mock the same API in tests with MSW. Write MSW handlers for the same URLs, so component tests get predictable data without a running server.

Step 6: Fetch real reference FX rates (no key). On the server, call Frankfurter once a day and cache it:

```
// app/api/fx/reference/route.ts
export const revalidate = 86400; // refresh once a day
export async function GET() {
  const res = await fetch("https://api.frankfurter.dev/v2/rates?base=gbp");
  if (!res.ok) return Response.json({ error: "FX source unavailable" }, { status: 502 });
  return Response.json(await res.json());
}
```

Keep a saved copy of one response in `src/mocks/data/fx-fallback.json` so the FX desk still works if the service is down. That's resiliency, which you can talk about in interviews.

Step 7: Make prices tick with a simulator. Start from each reference rate, then several times a second nudge it by a tiny random amount (e.g. ±0.02%), and work out a bid and ask price around it. Stream the ticks to the browser over WebSocket or Server-Sent Events, and batch them into AG Grid. Label the panel clearly: "Simulated prices, not for trading".

Step 8 (optional): Add a bank-connection demo with a sandbox. Sign up free for the TrueLayer or Plaid developer sandbox, connect to their mock bank with the test login they publish, and show the imported fake transactions. Keep the API keys in server-side environment variables (`.env.local` , listed in `.gitignore`) and never request live access.

Launch rules for a demo bank

- Show a clear banner on every page: **"SterI is a portfolio demo. Not a real bank. All data is fake."**
- Never ask visitors for real names, card numbers, bank logins or money. Use demo logins you create.
- Never commit API keys to GitHub. Use environment variables, and set them in Vercel's settings.
- In the README, explain what would change for a real launch (FCA authorisation, a real ledger, real Confirmation of Payee). Knowing the gap is part of thinking like a bank.

How you work: engineering process NEW

The posting asks about the SDLC, agile, CI/CD and AI tooling as much as about code. Make your process visible in the repo, because reviewers read it.

Practice	What to do	Where it shows
Agile board	A GitHub Projects board; write each feature as a user story ("As a risk officer, I want to see breaches so that…") with acceptance criteria	GitHub Projects

Practice	What to do	Where it shows
Branches and pull requests	One branch per story, a pull request with a description, CI must pass before merging; tag weekly releases (v0.1, v0.2…)	Pull request history, Releases
CI from day one	GitHub Actions: lint, typecheck, Vitest, build. Add a Java job (Maven/Gradle build + JUnit) in week 2. Status badge in the README	`.github/workflows/ci.yml`
Design artifacts	Short ADRs: 001 integer pence, 002 fake data only, 003 Java API + Next.js front end, 004 exposure calculation. A system diagram (front end, API, database, FX simulator)	`docs/adr/` , `docs/architecture.md`
AI-assisted development	Use Claude Code for code, tests, refactoring and docs. Log what the AI produced, what you checked, and what you fixed. Never accept AI code without tests and your own review	`AI_USAGE.md` , pull request checklist
Responsible AI	No secrets or sensitive data in prompts; Ask Sterl only reads an allow-list of fake fields; treat user questions as untrusted input; check AI answers against SQL results	`docs/responsible-ai.md`

```
# pull request checklist (.github/pull_request_template.md)
- [ ] Tests added or updated, CI green
- [ ] Money handled in integer pence
- [ ] No secrets, no real personal data
- [ ] AI-generated code reviewed line by line and logged in AI_USAGE.md
```

Build plan: four weeks to launch

Build Sterl during the 2 project hours of each sprint day. Week 1 builds the front end on fake data, week 2 adds the Java back end, week 3 adds the Credit Risk and FX desks, and week 4 makes it secure, deployed and ready to show. The data stays fake throughout.

> **If you fall behind,** protect in this order: Java API + SQL → Credit Risk desk → CI and tests → deploy. The FX desk, Ask Sterl and the extra features can follow after launch.

Week 1: front end on fake data, CI on day one

Day	What you build in Sterl	Skill practised
1–2	✅ Done: Next.js + TypeScript set-up, demo banner, money helpers in integer pence (`formatGBP` , `roundUp` , `toPence`) with 27 Vitest tests	TDD, TypeScript

Day	What you build in Sterl	Skill practised
3	GitHub Actions CI (lint, typecheck, tests, build); Projects board with week 1–4 user stories; pull request template; seeded Faker generator for accounts, 500 transactions and payees	CI/CD, agile
4	Zustand store: selected account, card frozen, theme, accessibility mode	Zustand
5	Mock API route handlers with delays and errors; load data with TanStack Query; optimistic card freeze	TanStack Query, resiliency
6	App Router pages: dashboard, accounts/[id], transactions, send money, cards, credit risk, markets; Server vs Client Components	Routing, layouts
7	Review day: ADR-001 and ADR-002, first architecture diagram, tag release v0.1, deploy to Vercel	Design artifacts, shipping

Week 2: Java + Spring Boot + PostgreSQL

Day	What you build in Sterl	Skill practised
8	Spring Boot project (`sterl-api`), health endpoint, JUnit set-up, Java job added to CI	Java, Spring Boot
9	PostgreSQL schema with Flyway migrations: accounts, double-entry ledger, payees, audit log; seed from the Faker JSON	SQL, data modelling
10	REST endpoints for accounts and transactions; filtering and pagination written in SQL	REST, SQL queries
11	Transfers with idempotency keys and ledger entries that always balance; JUnit tests proving it	Correctness, resiliency
12	Switch the front end from mock routes to the Java API; send money form with React Hook Form + Zod and the simulated payee name check	Integration, validation
13	Cards and round-ups through the API; every sensitive action written to the audit log	Auditability
14	Review day: ADR-003 (Java API + Next.js), update the diagram, release v0.2	Design artifacts

Week 3: Credit Risk desk + FX desk

Day	What you build in Sterl	Skill practised
15	Generate counterparties, facilities and drawdowns; tables and migrations in PostgreSQL	Data modelling

Day	What you build in SterI	Skill practised
16	`ExposureService` in Java: exposure, utilisation, breach detection; JUnit edge-case tests; ADR-004	Algorithms, testing
17	SQL reports (top exposures, breaches, exposure by sector and rating) exposed as API endpoints	SQL, reporting
18	Credit risk dashboard: AG Grid with utilisation bars, counterparty drill-down, concentration charts, CSV export, breach alerts	Data visualisation
19	FX desk: price simulator streaming into AG Grid with batched updates; Frankfurter reference rates with fallback	Real-time performance
20	React Testing Library tests for forms and the risk grid; Playwright tests for send money and breach review	Testing
21	Review day: Lighthouse 90+, accessibility audit, release v0.3	Quality

Week 4: secure, AI, cloud, launch

Day	What you build in SterI	Skill practised
22	Spring Security login for demo users with two-factor authentication; roles (customer, risk officer, admin)	Security
23	Ask SterI with guardrails: field allow-list, input checks, answers checked against SQL; `docs/responsible-ai.md`	Responsible AI
24	Docker for the Java API; deploy API + PostgreSQL to AWS free tier with a budget alert; front end on Vercel	Cloud
25	Observability: structured logs, health checks, error tracking, a simple alert on API errors	Operational stability
26	Security and code-quality pass: dependency audit, secrets scan, finish `AI_USAGE.md`	Secure SDLC
27	**Launch v1.0**: README with screenshots, architecture, CI badge, "bank thinking" and "Data and regulation" sections	Shipping
28	Job prep: 5-minute walkthrough video, CV update, apply	Communication

After launch: keep improving (still fake money)

Phase	What you add	Skills it proves
Events	Credit rating downgrades and fraud alerts as events (e.g. Kafka or Redis streams) pushed to the UI in real time	Event-driven architecture

Phase	What you add	Skills it proves
Fraud	Rules engine for the fraud alert simulator	Business rules, testing
Risk analytics	Stress test: "what if every BB-rated borrower is downgraded?", showing the new breaches	Credit risk thinking, SQL
Platform	Infrastructure as code (Terraform), containers for the whole stack	Cloud, automation
Open banking demo (optional)	TrueLayer or Plaid sandbox connection to a mock bank (Step 8)	Third-party API integration

Getting hired at JPMorgan

Candidate reports describe the same path for most technology roles: a timed coding test, a recorded video interview, then a final "Super Day". The exact steps vary by programme and region.

Stage	What happens	How to prepare
1. Application	CV through JPMorgan careers; graduate programme or experienced role	Lead with Sterl: live link, GitHub, 2–3 results with numbers
2. HackerRank test	Timed coding problems, usually easy to medium (sliding window and string/array problems are reported)	LeetCode easy/medium daily; practise under a timer
3. HireVue video	Recorded answers, sometimes explaining your code approach	Practise explaining solutions out loud, on camera
4. Super Day	Back-to-back technical and behavioural interviews	Sterl deep-dive, Java and React questions, SQL, system design, live coding

To give yourself the best chance:

- Apply to Java / React / AI roles like the Glasgow Wholesale Credit Risk one, and to the graduate Software Engineer Program or Code for Good if your professional experience is short
- Learn the basics of credit risk (exposure, limits, ratings, concentration) and of trading (orders, FX pairs, bid/ask prices)
- Practise SQL (joins, GROUP BY, window functions) and core Java (collections, streams, OOP) alongside LeetCode
- Prepare a 5-minute Sterl walkthrough covering one hard decision: exposure calculation, real-time performance, money accuracy, security, how you validated AI-written code, or why you used fake data

- Apply to other UK banks and fintechs in parallel (Barclays, HSBC, Lloyds, Monzo, Starling, Revolut) since one application is a long shot

Sources

- JPMorganChase: Software Engineer III – Java / React / AI, Wholesale Credit Risk, Glasgow (LinkedIn posting 4476977663, screenshots provided)
- JPMorgan: Software Engineer – JavaScript / React Web UI, London
- JPMorgan: Software Engineer III (UI), DMET Execute, London
- JPMorgan: Sr Lead Software Engineer – UI Developer – React, London
- JPMorgan: Software Engineer III – Expert UI Developer (React JS)
- Chase UK: Round-up account
- Chase UK app on the App Store
- Money to the Masses: Chase UK review
- TechPrep: JPMorgan's interview process (2026)
- Glassdoor: J.P. Morgan software engineer interviews
- Frankfurter: free exchange rates API, no key required
- Faker.js: finance module
- Finexer: Open banking sandboxes in the UK compared
- TrueLayer: Test payments in sandbox